PneumoScope-XAI

The complete project, explained

3-class pneumonia detection from chest X-rays (Normal / Bacterial / Viral)
with explainable and trustworthy AI

Deployed model	EfficientNetV2-B0 224px (fine-tuned)
Test accuracy / macro-F1	81.8% / 0.809 (837 X-rays from unseen patients)
Normal vs Pneumonia	96.3% accuracy, AUC 0.993
Conformal prediction (90%)	single-label answer for 76% of X-rays, 87.1% correct on those
Models compared	11 (classical ML, CNN, DenseNet121, EfficientNetV2 at 3 resolutions)
Author	Ankit Singh
Repository	github.com/sumankit/PneumoScope-XAI
Date	15 October 2026

Educational project. Not a medical device and not for clinical use.

Contents

How to read this guide 4

1. The project in one page 5

1.1 The problem . . . 5

1.2 Why it is hard and why it is different . . . 5

1.3 What was built . . . 5

1.4 Final results in one table . . . 5

1.5 The five things to remember . . . 6

2. How the whole system fits together 7

2.1 The life of one X-ray in the app . . . 8

3. Data: where it comes from and how it is prepared 9

3.1 The dataset . . . 9

3.2 Preprocessing: every step and why . . . 9

4. The models: what each one is and why it is here 11

4.1 Classical machine learning . . . 11

4.2 Convolutional neural networks: the building blocks . . . 12

4.3 Transfer learning and two-stage fine-tuning . . . 12

4.4 DenseNet121 . . . 13

4.5 EfficientNetV2-B0 (new in this round) . . . 14

4.6 How the deployed model is chosen . . . 14

5. How the models are trained 15

6. How the models are judged 16

6.1 The metrics, in plain words . . . 16

6.2 Confusion matrix of the deployed model . . . 16

6.3 Is a difference real? Confidence intervals and McNemar's test . . . 16

6.4 All models side by side . . . 17

6.5 Where the difficulty really is . . . 17

7. Resolution study: does a bigger image help? 18

7.1 What we learned . . . 19

8. Making the model trustworthy 20

8.1 Temperature scaling (calibration) . . . 20

8.2 Selective prediction ('refer to radiologist') . . . 20

8.3 Conformal prediction (experiment C) . . . 20

8.4 Grad-CAM: where did the model look? . . . 22

9. The hidden GPU bug: how it was found and fixed 24

9.1 Symptom . . . 24

9.2 Investigation 24
9.3 Impact on the original models 24
9.4 Fix 24
10. The application **25**
11. Key design decisions (and the reason for each) **26**
12. Limitations and future work **27**
12.1 Limitations 27
12.2 Future work 27
13. Interview and viva questions, with answers **28**
14. How to run the project **30**
15. Glossary **31**

How to read this guide

This guide explains **every part** of the project: each model, each method and each design decision. Each one is explained with the same four questions:

- **What it is**: the idea in plain words.
- **How it works**: the mechanism, step by step.
- **Why we used it**: the reason it is in *this* project, not just in general.
- **What it gave us**: the actual result in this project.

All numbers come from the project's own result files and were produced on the author's MacBook Air (Apple M5). The full technical reference (every file, every function, the maths) is `docs/PROJECT_REPORT.md`. This PDF is the explanation you read *first*, and the one to use for viva and interview preparation. The last chapters contain likely interview questions with answers.

1. The project in one page

1.1 The problem

A chest X-ray has to be classified into one of three classes. **Normal**: healthy lungs. **Bacterial pneumonia**: usually one dense white area, called a *consolidation*. **Viral pneumonia**: usually fine, diffuse streaks spread over both lungs, called an *interstitial pattern*. The distinction matters because bacterial pneumonia is treated with antibiotics and viral pneumonia is not.

1.2 Why it is hard and why it is different

- Most online projects solve only **Normal vs Pneumonia**, which is almost solved: here even a simple HOG + logistic regression reaches 93.8%. This project solves the much harder **3-class** problem.
- The dataset has several X-rays of the same child. Many projects split images randomly, so the same child appears in training and testing and the scores are inflated. This project splits by **patient**.
- A medical tool needs more than accuracy. It must show **why** it decided, give **honest confidence**, and know **when not to answer**.

1.3 What was built

Component	One-line description
Leakage-free dataset	5,856 X-rays relabelled into 3 classes, split 4,182 / 837 / 837 so that no patient is in two splits
11 models	HOG + Logistic Regression, HOG + Random Forest, CNN from scratch, DenseNet121 and EfficientNetV2-B0 at 224 / 384 / 512 px, each transfer model in a frozen and a fine-tuned version
Fair evaluation	same test set and metrics for every model, bootstrap confidence intervals, McNemar significance tests
Resolution study	does a bigger input image help EfficientNetV2? Accuracy vs compute cost
Trust layers	temperature scaling, 'refer to radiologist' abstention, conformal prediction sets, Grad-CAM + shortcut audit
Streamlit app	upload an X-ray, get a diagnosis, confidence, conformal set and heatmap

1.4 Final results in one table

Model	Test accuracy	Macro-F1	ROC-AUC
Logistic Regression (HOG)	77.1%	0.762	0.900
Random Forest (HOG)	78.0%	0.747	0.905
CNN (scratch)	81.2%	0.800	0.914
DenseNet121 (fine-tuned)	82.9%	0.816	0.930
EfficientNetV2-B0 224px (fine-tuned) (deployed)	**81.8%**	**0.809**	**0.916**
EfficientNetV2-B0 384px (fine-tuned)	83.4%	0.817	0.924
EfficientNetV2-B0 512px (fine-tuned)	82.6%	0.814	0.928

1.5 The five things to remember

- **Pneumonia detection is easy; the subtype is hard.** Normal vs Pneumonia: 96.3%. Bacterial vs Viral: 79.4%. 80% of all mistakes are bacterial↔viral.
- **All modern deep models tie** at about 81–83%. Differences between them are not statistically significant. The data, not the architecture, is the limit.
- **Bigger images did not help** the viral class (resolution study), but they cost up to 5.2× more compute.
- **Conformal prediction** turns the model into one that says "I'm sure: Bacterial" for 76% of X-rays, and "pneumonia, but I can't tell which type" when it isn't, with a 90% coverage guarantee.
- **A hidden GPU bug** (tensorflow-metal) was changing model outputs. It was found, explained, fixed, and every model was retrained.

2. How the whole system fits together

The diagram shows the full path from raw data to the app. Each box is one Python file or one stage. Arrows show which outputs feed which stage.

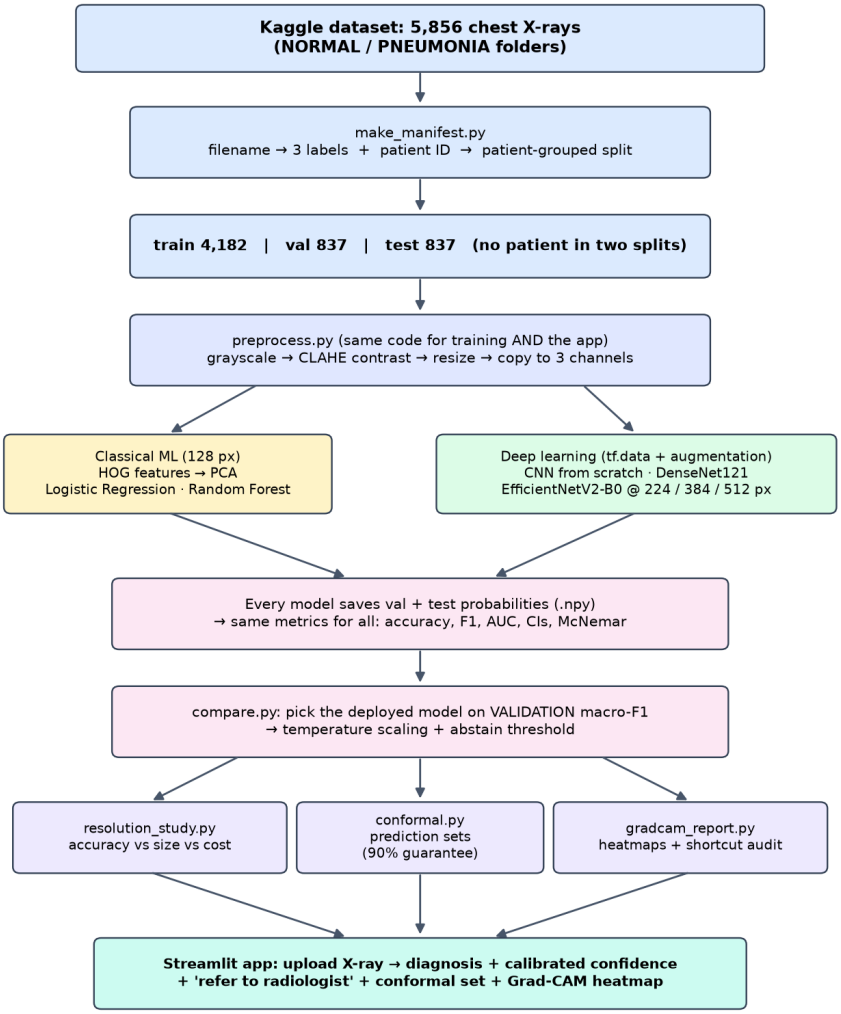


Figure 1. End-to-end pipeline.

2.1 The life of one X-ray in the app

Step	What happens	Code
1	You upload a JPG/PNG. It is read as a grayscale array.	app/streamlit_app.py
2	The same preprocessing as in training runs: CLAHE contrast, resize to the deployed model's size (224 px), copy into 3 channels.	src/data/preprocess.py
3	One forward + backward pass through the model gives both the 3 class probabilities and the Grad-CAM heatmap.	src/models/gradcam.py
4	Probabilities are temperature-scaled (T = 0.98) and shown as a bar chart.	src/models/calibration.py
5	If the top probability is below 0.70, the app says "Uncertain, refer to a radiologist".	app/inference.py
6	The conformal threshold for the chosen coverage (default 90%) gives the prediction set, e.g. {Bacterial, Viral}.	src/evaluation/conformal.py
7	The heatmap is overlaid on the X-ray, and a focus check warns if most attention is on the image border.	src/models/gradcam.py

3. Data: where it comes from and how it is prepared

3.1 The dataset

Kermany et al. (2018), Guangzhou Women and Children's Medical Center, children aged 1–5, licence CC BY 4.0. Kaggle mirror: *paultimothymooney/chest-xray-pneumonia*, downloaded with *kagglehub* (no API key needed). The original folders are NORMAL and PNEUMONIA. The pneumonia type is written in each filename, e.g. `person1234_bacteria_3131.jpeg`.

Split	Normal	Bacterial	Viral	Total	Used for
train	1,130	1,985	1,067	4,182	learning the weights
val	227	397	213	837	early stopping, choosing the deployed model, calibration, thresholds
test	226	398	213	837	final evaluation, touched once

Patient-grouped, stratified split (StratifiedGroupKFold)	
What it is	A way of splitting data so that all X-rays of one patient land in the same split (*grouped*), while each split keeps the same class proportions (*stratified*).
How it works	The patient ID is read from the filename (person1234 → PERSON1234, IM-0115-0001 → IM-0115). scikit-learn's StratifiedGroupKFold is run twice: once with 7 folds to cut off ≈15% as test, once with 6 folds on the rest to cut off ≈15% as validation. Afterwards a hard check (`assert leaked == 0`) stops the program if any patient appears in two splits.
Why we used it	With a random split, near-identical X-rays of the same child end up in train and test, and the model gets credit for recognising the child instead of the disease. The original Kaggle validation folder also has only 16 images, which is useless.
What it gave us	5,856 images in 3,118 patient groups, with zero leakage. One finding: the dataset reuses 'personN' numbers for *different* children (person255_bacteria and person255_virus are visibly different kids). We group on the bare number anyway. That is slightly over-cautious, but it guarantees that no child is ever split.

3.2 Preprocessing: every step and why

Grayscale + CLAHE (Contrast-Limited Adaptive Histogram Equalisation)	
What it is	A contrast-enhancement technique that works on small tiles of the image instead of the whole image at once.
How it works	The image is cut into an 8×8 grid of tiles. In each tile the brightness histogram is stretched, so dark and bright regions both show detail. The 'clip limit' (2.0) caps the stretching so noise isn't amplified. Neighbouring tiles are blended smoothly.
Why we used it	X-rays come from different machines and exposures, some too dark, some washed out. CLAHE makes them look more alike and makes lung texture (exactly what separates viral from bacterial) easier to see.
What it gave us	Applied identically in training and in the app via one shared function (preprocess.py). One CLAHE object per thread was needed because OpenCV's CLAHE crashed when shared between the parallel data-loading threads.

Resize + 3 channels + normalisation inside the model	
What it is	Every image becomes a fixed-size, 3-channel picture, which is what the networks expect.
How it works	Resize with INTER_AREA (good for shrinking) to 224 px (CNN, DenseNet), 224/384/512 px (EfficientNetV2) or 128 px (HOG). The single gray channel is copied 3 times. Values stay in 0–255; each model's first layers rescale them (÷255 and ImageNet mean/std) *inside* the model.
Why we used it	ImageNet-pretrained networks expect RGB input. Putting normalisation inside the model means the app can never apply the wrong normalisation, a common deployment bug.
What it gave us	The app needs no model-specific preprocessing code. Switching the deployed model from DenseNet to EfficientNetV2 needed no change in the app.

Data augmentation (training only)	
What it is	Random, realistic changes to training images so the model sees more variety than the dataset contains.
How it works	Each training batch gets: rotation ±10°, zoom ±10%, shift ±5%, brightness ±10%, contrast ±10% (Keras preprocessing layers). Validation and test images are never augmented.
Why we used it	There are only ~4,200 training images. Augmentation reduces overfitting. **No horizontal flip**: the heart is on the left, so a mirrored chest is not a realistic X-ray.
What it gave us	Training accuracy and validation accuracy stay close (see training curves), so the models generalise instead of memorising.

Class weights	
What it is	Giving mistakes on rare classes a larger penalty during training.
How it works	weight(class) = total images ÷ (3 × images of that class) → Normal ≈ 1.23, Bacterial ≈ 0.70, Viral ≈ 1.31. Each image's loss is multiplied by its class weight.
Why we used it	There are almost twice as many bacterial as viral images. Without weights the model would lean towards 'bacterial'. Weights were chosen over oversampling, which duplicates images and invites memorising them.
What it gave us	Viral recall stays reasonable (0.64 for the deployed model) despite viral being the minority class.

tf.data pipeline with caching	
What it is	TensorFlow's input pipeline that loads, transforms and batches images in parallel with training.
How it works	Paths → parallel decode + CLAHE + resize (OpenCV inside tf.numpy_function) → **cache** as small grayscale uint8 → shuffle (train only) → expand to 3-channel float → batch → augment → prefetch.
Why we used it	Decoding and CLAHE are slow. Caching them once (≈300 MB at 224 px, ≈1.5 GB at 512 px) makes every later epoch fast, while shuffling and augmentation still change every epoch.
What it gave us	Training throughput is limited by the GPU, not by image loading.

4. The models: what each one is and why it is here

The models form a ladder from simple to powerful. Each rung answers a question: *how far do hand-made features get? → can a small network learn better features itself? → does ImageNet knowledge help? → does a modern, efficient architecture or a larger image help more?*

4.1 Classical machine learning

HOG: Histogram of Oriented Gradients	
What it is	A hand-designed way to describe an image by the directions of its edges.
How it works	The 128×128 image is divided into 16×16-pixel cells. In each cell the gradient (edge) direction of every pixel is computed and counted into 9 direction bins. Neighbouring 2×2 cells are normalised together. Result: a 1,764-number vector per image.
Why we used it	It is the classic pre-deep-learning approach. It gives an honest baseline: anything a neural network achieves must be compared to it.
What it gave us	With Logistic Regression it reaches 77.1% accuracy, and already 93.8% on Normal vs Pneumonia. So edges alone mostly solve the easy task.

PCA: Principal Component Analysis	
What it is	A way to compress many correlated numbers into fewer uncorrelated ones while keeping most of the information.
How it works	It finds the directions in which the 1,764 HOG features vary the most and projects every image onto the top 256 of them.
Why we used it	Logistic regression on 1,764 correlated features with ~4k images overfits and is slow. 256 components keep the important variation.
What it gave us	Used inside the Logistic Regression pipeline: StandardScaler → PCA(256) → Logistic Regression.

Logistic Regression (multinomial)	
What it is	A linear classifier: a weighted sum of the features is turned into 3 class probabilities by the softmax function.
How it works	For each class it learns one weight per feature. Score(class) = w · features + b; softmax turns the 3 scores into probabilities. C = 0.1 (strong regularisation) and class_weight = 'balanced'.
Why we used it	The simplest probabilistic classifier. If it does well, the problem is easy; if not, we need non-linear models.
What it gave us	Accuracy 77.1%, macro-F1 0.762, Viral F1 0.597.

Random Forest	
What it is	An ensemble of many decision trees that vote.
How it works	400 trees are grown, each on a random bootstrap sample of the images and random subsets of features. Each tree asks yes/no questions about feature values. The forest's prediction is the share of trees voting for each class.
Why we used it	A strong non-linear classical model that needs almost no tuning. It is the 'best you can do without deep learning' baseline.
What it gave us	Accuracy 78.0%, but macro-F1 only 0.747, because it struggles most with viral (F1 0.532). It also has the worst calibration (ECE 0.113).

4.2 Convolutional neural networks: the building blocks

Building block	What it does (plain words)
Convolution (Conv2D)	Slides small 3×3 filters over the image. Each filter learns to detect one pattern (an edge, a texture, a blob). Early layers find edges; deeper layers combine them into shapes like 'white patch in lower lung'.
ReLU / SiLU activation	Keeps positive signals and suppresses negative ones, so the network can learn non-linear patterns.
Max pooling / stride	Shrinks the feature map, keeping the strongest responses. It makes the network look at bigger regions as it goes deeper.
Batch Normalisation	Rescales each layer's outputs to a standard range during training. Training is faster and more stable.
Global Average Pooling	Averages each feature map into one number. Turns 'where + what' into 'what', and works at any image size.
Dense + Dropout	A fully connected layer combines features. Dropout randomly switches off 50% of neurons while training so the network can't rely on a few of them, which reduces overfitting.
Softmax	Turns the final 3 scores into probabilities that sum to 1.

CNN from scratch	
What it is	A small convolutional network designed and trained by us, starting from random weights.
How it works	4 blocks of [Conv 3×3 → BatchNorm → ReLU → MaxPool] with 32, 64, 128 and 256 filters → Global Average Pooling → Dense(128) → Dropout(0.5) → Softmax(3). Trained with Adam (lr 1e-3) for up to 50 epochs.
Why we used it	Shows that we understand CNNs ourselves, and answers 'how much does pretraining on ImageNet actually add?'
What it gave us	Accuracy 81.2%, macro-F1 0.800: already clearly better than HOG, and statistically tied with the much bigger pretrained models. The task is limited by the data, not by model size.

4.3 Transfer learning and two-stage fine-tuning

Transfer learning	
What it is	Reusing a network trained on a huge dataset (ImageNet: 1.4 million everyday photos) as the starting point for our task.
How it works	The pretrained network's convolutional part (the *backbone*) already detects edges, textures and shapes. We remove its 1000-class ImageNet head and attach our own head: Global Average Pooling → Dense(256, ReLU) → Dropout(0.5) → Softmax(3).
Why we used it	With only ~4k X-rays, learning good low-level features from scratch is hard. Pretrained features give a big head start. CheXNet (Stanford) used the same idea with DenseNet121.
What it gave us	Both pretrained backbones beat our scratch CNN after fine-tuning, though not significantly (see Chapter 6).

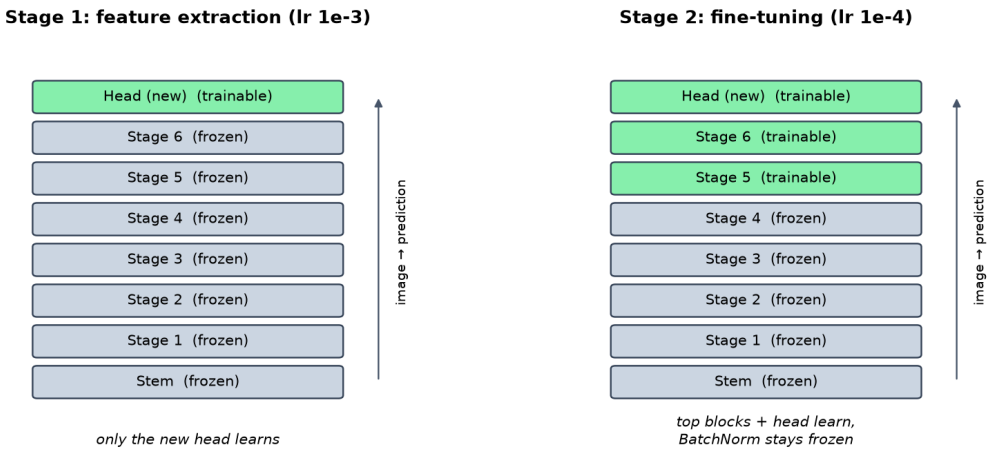


Figure 2. Two-stage training. Green = trainable, grey = frozen.

Two-stage training (feature extraction, then fine-tuning)	
What it is	First train only the new head, then also adapt the top part of the backbone.
How it works	**Stage 1**: backbone frozen, head trained with learning rate 1e-3. **Stage 2**: unfreeze the top blocks (DenseNet121: from conv4_block13; EfficientNetV2: from block5a) and continue with a 10× smaller learning rate (1e-4). **BatchNorm layers stay frozen** so their ImageNet statistics aren't destroyed by small batches.
Why we used it	If everything is unfrozen at once, the random new head sends large, noisy gradients into the pretrained layers and damages them. Two stages let the head settle first.
What it gave us	Fine-tuning helped every time, e.g. EfficientNetV2 224 px: frozen 78.6% → fine-tuned 81.8% (significant). An earlier DenseNet attempt with lr 1e-5 *underfitted* (training accuracy below validation). Raising it to 1e-4 fixed that, which is a good example of reading training curves.

4.4 DenseNet121

DenseNet121	
What it is	A 121-layer CNN (2017) in which every layer inside a 'dense block' receives the outputs of *all* earlier layers in that block.
How it works	Instead of adding a layer's input to its output (ResNet), DenseNet **concatenates** all previous feature maps. Each layer adds only a few new channels ('growth rate'), and features are reused heavily. It has 4 dense blocks with transition layers (1×1 conv + pooling) in between, about 7 million parameters.
Why we used it	The standard backbone in chest X-ray research (CheXNet). Feature reuse and strong gradient flow suit small medical datasets.
What it gave us	Accuracy 82.9%, macro-F1 0.816, best ROC-AUC (0.930). Statistically tied with the deployed model (McNemar p = 0.35).

4.5 EfficientNetV2-B0 (new in this round)

EfficientNetV2-B0	
What it is	A 2021 CNN family designed for high accuracy with few parameters and fast training. B0 is the smallest member (5.9 million parameters in the backbone, 6,248,019 with our head).
How it works	Stem conv → 3 stages of **Fused-MBConv** blocks (a regular 3×3 conv that expands channels, then a 1×1 conv that shrinks them; fast on GPUs for large early feature maps) → 3 stages of **MBConv** blocks (1×1 expand → 3×3 *depthwise* conv → **Squeeze-and-Excitation** → 1×1 project, with a skip connection) → 1×1 conv to 1,280 channels (top_activation). Squeeze-and-Excitation learns to give more weight to the most useful channels for each image. Activation is SiLU (x·sigmoid(x)).
Why we used it	(1) A modern, efficient architecture to compare against DenseNet. (2) It is fully convolutional and was designed for scaling resolution, which makes it ideal for the resolution study. (3) B0 is similar in size to DenseNet121, so the comparison is fair. V2-S was benchmarked too but would take ~5 h per run at 512 px on this laptop.
What it gave us	Fine-tuned at 224 px: 81.8% accuracy, macro-F1 0.809. It had the **highest validation macro-F1** (0.819) of all models, so it is the deployed model.

Stage	Block type	Blocks	Output at 224 px input
Stem	3×3 conv, stride 2	–	112×112×32
1	Fused-MBConv	1	112×112×16
2	Fused-MBConv	2	56×56×32
3	Fused-MBConv	2	28×28×48
4	MBConv + SE	3	14×14×96
5	MBConv + SE	5	14×14×112
6	MBConv + SE	8	7×7×192
Top	1×1 conv → BN → SiLU	–	7×7×1280 (12×12 at 384 px, 16×16 at 512 px)

Stages 5, 6 and the top layer (154 non-BatchNorm layers) are fine-tuned in stage 2.

4.6 How the deployed model is chosen

Rule, fixed before looking at test results: deploy the fine-tuned model with the highest **validation** macro-F1. Result: EfficientNetV2-B0 224px (fine-tuned) (val macro-F1 0.819). On the test set, EfficientNetV2 384 px happens to score slightly higher (83.4%), but choosing by test score would be cheating: the test set would no longer be unseen. The difference is not significant anyway ($p = 0.13$), and 224 px is the cheapest model to run.

5. How the models are trained

Cross-entropy loss (with class weights)	
What it is	The number the network tries to make small during training.
How it works	For each image, loss = −log(probability the model gave to the correct class), multiplied by that class's weight. Confident correct predictions give a loss near 0; confident wrong ones give a large loss.
Why we used it	The standard loss for classification. It directly rewards putting high probability on the right class.
What it gave us	Validation loss is also what early stopping and checkpointing watch.

Adam optimizer and learning rate	
What it is	The algorithm that updates the weights after each batch, and the step size it uses.
How it works	Adam keeps a running average of each weight's gradient and of its squared gradient, and takes a step of roughly lr × (average gradient ÷ its typical size), which adapts the step per weight.
Why we used it	Works well out of the box. A higher lr (1e-3) suits a new head; a lower lr (1e-4) suits gently adapting pretrained layers.
What it gave us	lr 1e-3 for stage 1 and the scratch CNN, 1e-4 for stage-2 fine-tuning.

Callbacks: EarlyStopping, ReduceLROnPlateau, ModelCheckpoint	
What it is	Automatic rules that watch validation loss during training.
How it works	**ReduceLROnPlateau**: if validation loss hasn't improved for a few epochs, multiply lr by 0.3. **EarlyStopping**: if it hasn't improved for 6 epochs (7 for the CNN), stop and restore the best epoch's weights. **ModelCheckpoint**: save the model every time validation loss reaches a new best.
Why we used it	Stops training before overfitting, without guessing the number of epochs in advance.
What it gave us	Example: EfficientNetV2 224 px stage 2 ran 17 epochs and the best was epoch 11 (validation loss 0.444). Every fine-tuning stage lowered validation loss compared with its frozen stage.

Run	Epochs run	Best epoch	Best val loss
CNN (scratch)	31	24	0.488
DenseNet121 stage 1	18	12	0.524
DenseNet121 stage 2	10	4	0.464
EfficientNetV2-B0 224 px stage 1	16	10	0.522
EfficientNetV2-B0 224 px stage 2	17	11	0.444
EfficientNetV2-B0 384 px stage 1	8	2	0.526
EfficientNetV2-B0 384 px stage 2	11	5	0.455
EfficientNetV2-B0 512 px stage 1	8	2	0.529
EfficientNetV2-B0 512 px stage 2	11	5	0.447

6. How the models are judged

6.1 The metrics, in plain words

Metric	Meaning	Deployed model
Accuracy	Share of X-rays classified correctly.	81.8%
Precision (per class)	Of the X-rays the model called X, how many really are X?	0.938 / 0.828 / 0.668 (N/B/V)
Recall (per class)	Of the X-rays that really are X, how many did the model find?	0.929 / 0.849 / 0.643 (N/B/V)
F1 (per class)	Balance of precision and recall (harmonic mean).	0.933 / 0.839 / 0.656 (N/B/V)
Macro-F1	Average of the 3 F1 scores, so the small viral class counts as much as the others. The main metric, because classes are imbalanced.	0.809
ROC-AUC (one-vs-rest)	Probability that a random X-ray of a class is scored higher for that class than a random X-ray of another class (1.0 perfect, 0.5 random). Averaged over the 3 classes.	0.916
ECE (calibration error)	Average gap between how confident the model says it is and how often it is right. Lower is better.	0.050

6.2 Confusion matrix of the deployed model

True ↓ / Predicted →	Normal	Bacterial	Viral
Normal	210	3	13
Bacterial	5	338	55
Viral	9	67	137

Read row by row: of 213 viral X-rays, 137 were called viral and 67 were called bacterial. 122 of the 152 mistakes (80%) are bacterial↔viral confusions. Only 14 of 611 pneumonia cases were called normal (missed-pneumonia rate 2.3%).

6.3 Is a difference real? Confidence intervals and McNemar's test

Bootstrap 95% confidence interval	
What it is	A range that shows how much a score could change with a different sample of test patients.
How it works	The 837 test results are re-sampled with replacement 1,000 times, and the metric is computed on each re-sample. The middle 95% of those values is the interval.
Why we used it	837 test images is not huge. A single number like 81.8% hides an uncertainty of roughly ±2.5 points.
What it gave us	Deployed model accuracy 81.8% with 95% CI 79.3%–84.3%. Intervals of the top models overlap heavily.

McNemar's test	
What it is	A statistical test of whether two models really differ, using the same test images.
How it works	Count the images where model A is right and B is wrong (n01), and where B is right and A is wrong (n10). If both models were equally good, these counts should be similar. An exact binomial test gives the p-value; $p < 0.05$ means 'the difference is unlikely to be luck'.
Why we used it	Comparing accuracies alone can't tell a real improvement from noise.
What it gave us	Deep models vs HOG baselines: $p < 0.01$, a real improvement. Among fine-tuned deep models: $p \geq 0.13$, i.e. a statistical tie.

6.4 All models side by side

Model	Val macro-F1	Test accuracy	Test macro-F1	Viral F1	p vs deployed
Logistic Regression (HOG)	0.748	77.1%	0.762	0.597	0.00057
Random Forest (HOG)	0.754	78.0%	0.747	0.532	0.0027
CNN (scratch)	0.787	81.2%	0.800	0.643	0.65
DenseNet121 (frozen)	0.778	80.6%	0.785	0.618	0.35
DenseNet121 (fine-tuned)	0.816	82.9%	0.816	0.662	0.35
EfficientNetV2-B0 224px (frozen)	0.770	78.6%	0.774	0.619	0.0061
EfficientNetV2-B0 224px (fine-tuned)	**0.819**	**81.8%**	**0.809**	**0.656**	**—**
EfficientNetV2-B0 384px (frozen)	0.762	80.2%	0.786	0.635	0.18
EfficientNetV2-B0 384px (fine-tuned)	0.805	83.4%	0.817	0.654	0.13
EfficientNetV2-B0 512px (frozen)	0.760	78.6%	0.768	0.595	0.0055
EfficientNetV2-B0 512px (fine-tuned)	0.809	82.6%	0.814	0.667	0.53

6.5 Where the difficulty really is

Model	Normal vs Pneumonia acc	AUC	Bacterial vs Viral acc
Logistic Regression (HOG)	93.8%	0.983	74.8%
CNN (scratch)	95.5%	0.987	79.1%
DenseNet121 (fine-tuned)	97.3%	0.993	80.2%
EfficientNetV2-B0 224px (fine-tuned)	96.3%	0.993	79.4%
EfficientNetV2-B0 384px (fine-tuned)	96.9%	0.994	81.2%

Every model is close to perfect at spotting pneumonia (AUC ≈ 0.99) and stuck around 75–81% at telling the two types apart. Telling bacterial from viral on one X-ray is also hard for radiologists, and the labels come from clinical records, not from the image itself.

7. Resolution study: does a bigger image help?

Resolution study (experiment A)	
What it is	A controlled experiment: train the same EfficientNetV2-B0 three times, changing **only** the input size (224, 384, 512 px).
How it works	Same data split, preprocessing, augmentation, two-stage recipe, batch size (16), class weights and seed. For each run we measure accuracy, per-class F1, McNemar vs 224 px, FLOPs per image, inference time, time per epoch, total training time and peak memory.
Why we used it	**Hypothesis:** viral pneumonia shows as fine streaks that might be washed out when a 1,000–2,000 px X-ray is shrunk to 224 px. If so, bigger inputs should mainly help the viral class.
What it gave us	The hypothesis was **not supported**: viral F1 stayed flat, gains were not significant, and the compute cost rose sharply.

Input	Test acc	Macro-F1	Viral F1	Bact. vs Viral acc	p vs 224	GFLOPs / img	ms / img (b32)	Train time
224 px	81.8%	0.809	0.656	79.4%	—	1.46	5.9	37 min
384 px	83.4%	0.817	0.654	81.2%	0.13	4.27	11.1	45 min
512 px	82.6%	0.814	0.667	81.0%	0.53	7.59	20.2	60 min

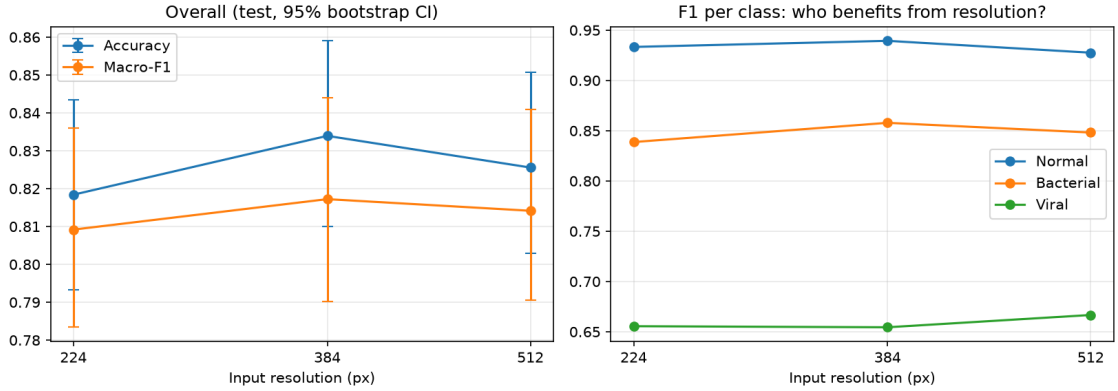


Figure 3. Accuracy (with 95% CI) and per-class F1 vs input resolution.

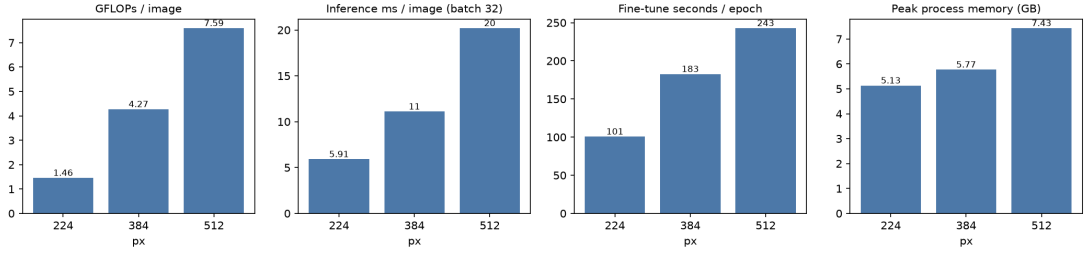


Figure 4. Compute cost vs resolution (measured on the Apple M5).

7.1 What we learned

- Compute grows with the square of the side length: 384 px costs 2.9× and 512 px 5.2× the FLOPs of 224 px.
- Viral F1: 0.656 → 0.654 → 0.667. The class we hoped to help did not improve.
- 384 px is the best test point, but the gain is within noise ($p = 0.13$). Validation actually preferred 224 px.
- Single-image response time is about 95 ms at every resolution, because it is dominated by fixed per-call overhead on the GPU. In batches the difference is real (5.9 → 20.2 ms per image).
- Conclusion: at this data size, resolution beyond 224 px is not worth the cost. The bottleneck is data and label noise, not image detail.

8. Making the model trustworthy

8.1 Temperature scaling (calibration)

Temperature scaling	
What it is	A one-number fix that makes the model's confidence values honest without changing its predictions.
How it works	All output scores (logits) are divided by a temperature T before the softmax. T > 1 makes the model less confident; T < 1 more confident. T is fitted on the validation set by minimising the negative log-likelihood. The ranking of classes never changes, so accuracy is identical.
Why we used it	Neural networks are often over-confident. A doctor reading '95%' should be able to trust that it means about 95%.
What it gave us	T = 0.98, so the deployed model was already nearly calibrated (ECE 0.050 → 0.048). Earlier, for DenseNet, scaling even made test calibration slightly worse. That was reported as an honest negative result.

8.2 Selective prediction ('refer to radiologist')

Selective prediction / abstention	
What it is	Letting the model refuse to answer when its confidence is low, and hand those cases to a human.
How it works	On the validation set we search for the lowest confidence threshold at which the answered cases reach the target accuracy (we try 97%, 95%, 90%, 85%, and keep the strictest that still answers ≥ 50% of cases). In the app, any X-ray below the threshold gets 'Uncertain, refer to a radiologist'.
Why we used it	A real clinical assistant should handle easy cases and flag hard ones. Forcing an answer on every image is unsafe.
What it gave us	Threshold 0.70 (target 90% on validation). On test: answers 73.1% of X-rays at 88.1% accuracy (vs 81.8% when forced to answer everything).

8.3 Conformal prediction (experiment C)

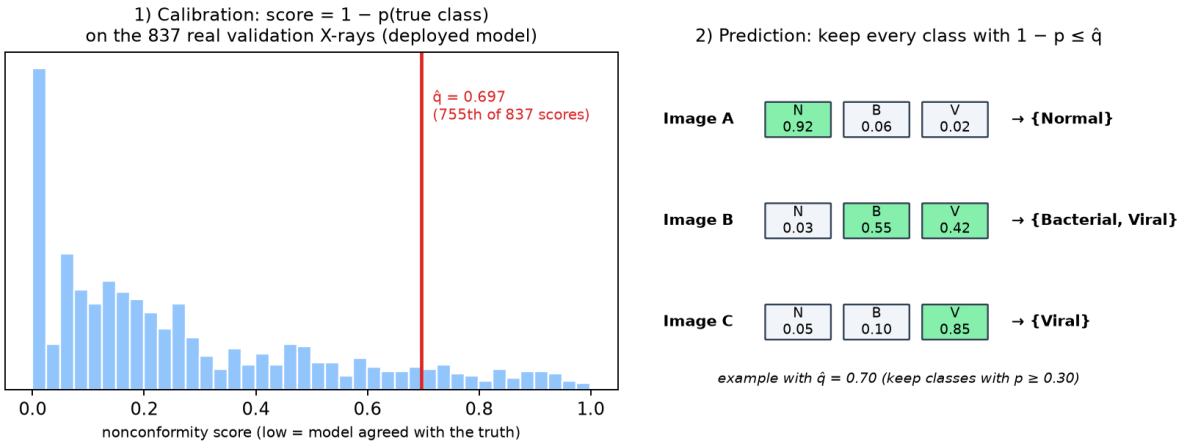


Figure 5. Left: the real calibration scores of the deployed model and the resulting threshold. Right: how the threshold turns probabilities into a prediction set (example probabilities chosen to illustrate the rule).

Split conformal prediction	
What it is	Instead of one label, the model returns a **set** of labels that contains the true one with a **guaranteed** probability (here 90%).
How it works	**1. Score:** for each validation X-ray compute how 'surprised' the model is by the true label: score = 1 − p(true class). **2. Threshold:** sort the 837 scores and take the ⌈(837+1)×0.9⌉ = 755th smallest. That is q̂. **3. Predict:** for a new X-ray include every class whose score 1 − p is ≤ q̂.
Why we used it	It is the only method here with a mathematical guarantee, and it needs no retraining. It fits the problem: when the model can't tell bacterial from viral, the honest answer is '{Bacterial, Viral}'.
What it gave us	q̂ = 0.697. Test coverage 89.4%; over 500 random calibration splits 90.1% on average. Average set size 1.24; a single label for 76.2% of X-rays.

Why the guarantee holds (the intuition). If the new X-ray is like the calibration X-rays, its score is equally likely to land anywhere in the sorted list of 837 + 1 scores. The chance that it lands at or below position 755 is therefore at least 755 ÷ 838 ≈ 90.1%, and that is exactly the chance that the true class is in the set. The '+1' in the formula is the small-sample correction that makes the guarantee exact.

The four variants we compared

Method	Score / idea	Coverage	Avg size	Single label	Viral coverage
LAC (app)	1 − p(class); smallest sets	89.4%	1.24	76%	77.9%
APS	add classes from most to least likely until the mass reaches q̂	90.8%	1.52	48%	78.9%
APS randomized	APS with random tie-breaking: exact, less conservative	91.0%	1.40	62%	81.2%
Class-conditional LAC	one threshold per class, so each class gets 90%	90.1%	1.33	67%	86.4%

Set returned (LAC, 90%)	Share of test X-rays
{Bacterial}	35.5%
{Normal}	25.2%
{Bacterial, Viral}	21.4%
{Viral}	15.5%
{Normal, Viral}	2.3%
{Normal, Bacterial}	0.1%

What the conformal results taught us

- **The guarantee works on average.** 500 random re-splits give 90.1% mean coverage for a 90% target.
- **An average can hide a weak class.** LAC covers Normal 95.1%, Bacterial 92.2%, but Viral only 77.9%.
- **Class-conditional conformal fixes it**: over random splits every class gets ≈ 90.2%. On the fixed val→test split viral reaches 86.4%, because the validation set was also used to pick the model and therefore looks slightly easier than truly new data. Fix for the future: a separate calibration split.
- **Better models earn the same guarantee with smaller sets.** Logistic Regression needs about 1.5 labels per X-ray, the deep models about 1.2.

8.4 Grad-CAM: where did the model look?

Grad-CAM (Gradient-weighted Class Activation Mapping)	
What it is	A heatmap showing which regions of the X-ray pushed the model towards its prediction.
How it works	Take the last convolutional feature maps (EfficientNetV2: *top_activation*, 7×7×1280 at 224 px). Compute how much the predicted class's score changes when each feature map changes (gradients). Average those gradients per map to get its importance, add up the maps weighted by importance, keep only positive values (ReLU), and stretch the result to image size. Red = high influence.
Why we used it	In medicine a prediction without a reason isn't trustworthy. Grad-CAM shows whether the model looks at the lungs or at something irrelevant.
What it gave us	Shown in the app for every prediction, and analysed over the whole test set (below).

Border-attention audit (shortcut-learning check)	
What it is	A number that measures how much of the heatmap lies near the image edges, where hospital text, 'R' markers and tubes appear.
How it works	Share of heatmap mass inside the outer 12% of the image. The strip covers ~42% of the area, so a uniform heatmap scores ≈ 0.42; lower means more focus on the chest.
Why we used it	Models sometimes 'cheat' by learning hospital-specific marks instead of disease (shortcut learning). The audit detects it automatically.
What it gave us	Deployed model: 0.322 overall. Bacterial 0.247 (focused on the lungs), Viral 0.357, Normal 0.423. DenseNet shows the same viral pattern (0.378).

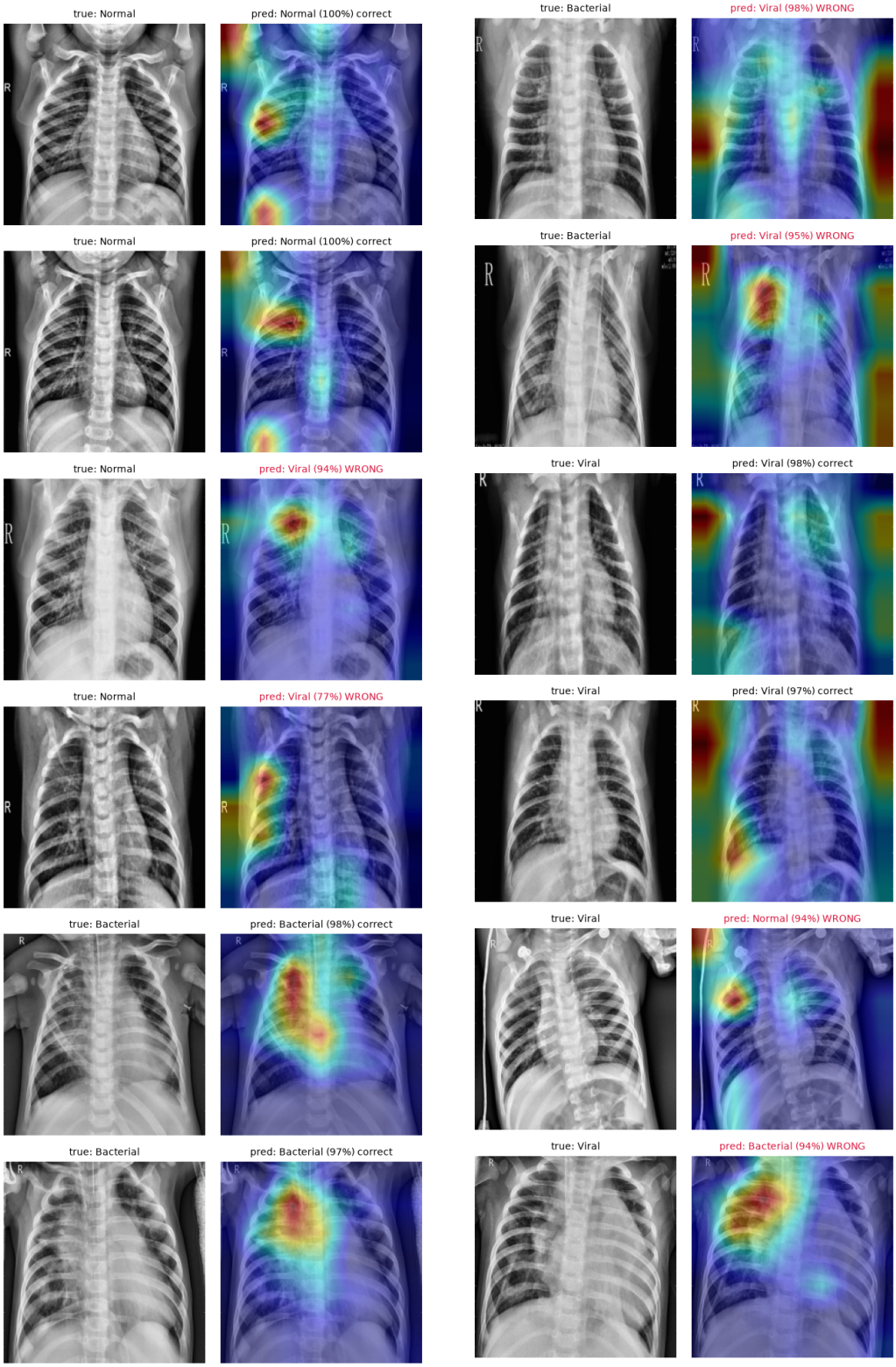


Figure 6. Grad-CAM of the deployed model: for each true class, two confident correct and two wrong predictions.

Honest finding. Bacterial predictions point at the consolidated lung region, which is correct anatomy. Several high-confidence **viral** predictions put most of the heat on the dark side margins *outside* the body. Since the same pattern appears in two different architectures, it is probably a property of the dataset (e.g. viral X-rays framed or cropped differently), not of one model. Part of the 'viral' signal may therefore not be medical. This is the project's main open limitation, and a lung-segmentation mask is the planned fix.

9. The hidden GPU bug: how it was found and fixed

This is the best 'engineering story' in the project, and a strong interview answer to *'tell me about a hard bug you solved'*.

9.1 Symptom

The app's demo images were chosen as the model's most confident *correct* test predictions (~96–97%). In the app they showed only 72–79%. Same model, same image, different answer.

9.2 Investigation

Step	Test	Result
1	Compare the pixels the training pipeline produced with the pixels the app produced	Identical (difference 0.0), so preprocessing is not the cause
2	Run the same model on the same image via model(x) vs model.predict(x)	0.801 vs 0.969: the two calling styles disagree
3	Repeat on CPU only	0.801 for every calling style; CPU is the reference
4	GPU in 'eager' mode vs 'graph' mode	Eager 0.801 (correct), graph 0.969 (wrong)
5	Switch off TensorFlow graph optimizations one by one (11 passes)	None of them alone fixes it
6	Switch off the whole meta-optimizer (disable_meta_optimizer)	GPU graph = 0.801, matches CPU

Root cause: tensorflow-metal (the Apple-GPU plugin) adds its own graph optimizer that computes a numerically different function for these networks. Training, early stopping and evaluation all ran through that path; the app and any CPU server run the correct path. The published numbers therefore did not describe the model that users would actually run.

9.3 Impact on the original models

Model	Accuracy reported (wrong path)	Real accuracy (same weights)	Predictions that changed class
DenseNet121 (fine-tuned)	82.9%	81.0%	6.5%
EfficientNetV2-B0 224px (fine-tuned)	80.8%	77.3%	11.4%
EfficientNetV2-B0 384px (fine-tuned)	82.2%	82.2%	3.7%
EfficientNetV2-B0 512px (fine-tuned)	81.2%	80.4%	3.6%

9.4 Fix

- `src/utils/runtime.py`: `configure_tf()` disables the meta-optimizer on Apple Silicon before any training starts (costs 15–50% speed).
- All deep models (CNN, DenseNet121, three EfficientNetV2s) were **retrained from scratch**. Saved predictions now match a pure-CPU run to about 0.000001.
- The wrong pre-fix results are kept in `reports/metal_bug/` as evidence.
- Lesson: always check that the evaluation code path and the serving code path produce the same output for the same input.

10. The application

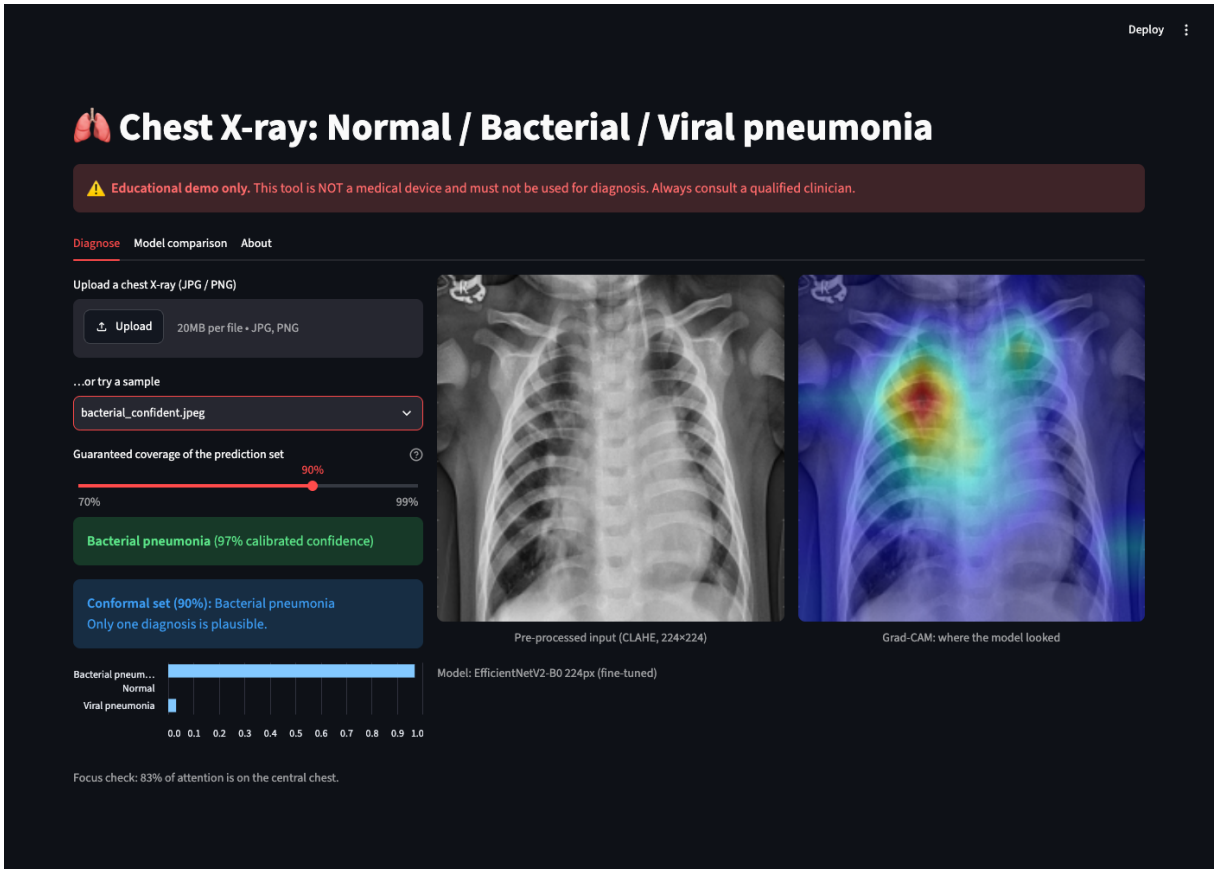


Figure 7. Diagnose tab: prediction, conformal set with coverage slider, probabilities, Grad-CAM.

Part	What the user sees	Behind it
Upload / sample	Upload a JPG/PNG or pick one of 4 demo X-rays	Image read with Pillow → grayscale array
Coverage slider	70%–99% guarantee level for the prediction set	Looks up the stored q̂ for that level (no recomputation)
Green / yellow box	Diagnosis with calibrated confidence, or 'Uncertain, refer to a radiologist'	Temperature scaling + abstain threshold from models_saved/*.json
Blue box	Conformal set, e.g. 'Bacterial pneumonia · Viral pneumonia'	predict_sets() with the stored q̂
Bar chart	Probability of each class	Calibrated softmax
Images	Pre-processed input and Grad-CAM overlay	gradcam() + overlay()
Focus check	Warning if > 50% of attention is near the border	border_attention()
Model comparison tab	All results, resolution study and conformal analysis	Reads reports/results/*.json

The model is loaded once per server (`@st.cache_resource`). Which model is served is read from `models_saved/deployed.json`, so retraining and re-running the comparison automatically updates the app. There is no separate REST API: Streamlit runs UI and inference in one Python process.

11. Key design decisions (and the reason for each)

Decision	Why
Split by patient, not by image	Prevents the model from recognising the same child in test; avoids inflated scores
One shared preprocessing function	Training and app can never preprocess differently
Normalisation inside the model	The app just passes 0–255 images; no model-specific code to get wrong
No horizontal flip	A mirrored chest is anatomically wrong (heart on the left)
Class weights instead of oversampling	Balances classes without duplicating images
Two-stage fine-tuning, BatchNorm frozen	Protects pretrained features from a random new head and from small-batch statistics
Macro-F1 as the main metric	The viral class is smaller but just as important
Choose the deployed model on validation, never test	Keeps the test score honest
Save probabilities, analyse later	All metrics, calibration and conformal analyses re-run in seconds without retraining
Model registry (one list of all models)	Training, reports and the app can't disagree about file names, sizes or Grad-CAM layers
EfficientNetV2-B0 rather than V2-S	Same size class as DenseNet, and a 3-resolution study fits on a laptop
Report negative results	Temperature scaling not helping, resolution not helping, viral shortcut: all stated openly

12. Limitations and future work

12.1 Limitations

- One hospital, children aged 1–5, one dataset. No external validation, so performance on adults or other hospitals is unknown.
- Bacterial/viral labels come from clinical records and are probably partly noisy.
- One training run per configuration. Differences under ~1–2 points are within noise (two identical 224 px runs differed by 0.2 points of accuracy and 0.6 of macro-F1).
- The validation set is used for early stopping, model choice, temperature, abstention and conformal calibration, which makes its estimates slightly optimistic (measured: viral conformal coverage 86% instead of 90%).
- Viral predictions partly rely on image borders (Grad-CAM audit).
- Not deployed publicly as part of this work.

12.2 Future work

- A separate calibration split for conformal prediction and thresholds.
- External test set (e.g. RSNA Pneumonia) to measure how well the model transfers to another hospital.
- Lung segmentation (U-Net) so the classifier only sees the lungs, then re-check the viral shortcut.
- Backbones pretrained on chest X-rays (e.g. TorchXRayVision, CheXpert) instead of ImageNet.
- Hierarchical model: first Normal vs Pneumonia, then a dedicated Bacterial vs Viral model.
- Several runs per configuration or a deep ensemble, for error bars and better uncertainty.

13. Interview and viva questions, with answers

Q. Why 3 classes and not the usual binary task?

Binary is nearly solved: even HOG + logistic regression gets 93.8%. Bacterial vs viral is the clinically relevant and genuinely hard part, so it shows more.

Q. What is data leakage here and how did you prevent it?

The same child has several X-rays. With a random split, one image is in training and a near-duplicate in test. I parsed patient IDs from filenames and used StratifiedGroupKFold, with an assert that no patient spans two splits.

Q. Why macro-F1 instead of accuracy?

The classes are imbalanced (bacterial is about twice viral). Accuracy can look good while viral is poorly detected. Macro-F1 weights all three classes equally.

Q. Why transfer learning, and why two stages?

With ~4k images, ImageNet features give a strong start. Stage 1 trains only the new head so its random gradients don't damage pretrained layers. Stage 2 unfreezes the top blocks with a 10× smaller learning rate. BatchNorm stays frozen to keep its statistics.

Q. Why EfficientNetV2-B0?

It is modern and efficient (Fused-MBConv + MBConv with Squeeze-and-Excitation), fully convolutional (any input size, so good for a resolution study), and similar in size to DenseNet121, which makes the comparison fair.

Q. Which model is best?

By the pre-defined rule (validation macro-F1), EfficientNetV2-B0 at 224 px, 81.8% test accuracy. But all fine-tuned deep models are statistically tied (McNemar $p \geq 0.13$), so the honest answer is that the data, not the architecture, limits performance.

Q. Did higher resolution help?

No. Viral F1 stayed about 0.65–0.67 at 224/384/512 px, the best gain was not significant, and 512 px costs 5.2× the FLOPs.

Q. What is conformal prediction in one sentence?

A method that turns any classifier's probabilities into a set of labels guaranteed to contain the true label with a chosen probability, calibrated on held-out data.

Q. How do you compute the conformal threshold (q-hat)?

Score each calibration image as $1 - p(\text{true class})$, sort the scores, and take the $\lceil (n+1)(1-\alpha) \rceil$-th smallest. With $n = 837$ and $\alpha = 0.1$ that is the 755th.

Q. Does conformal prediction need a calibrated model?

No. The guarantee holds for any model; a badly calibrated or weak model just produces bigger sets.

Q. What is the difference between temperature scaling and conformal prediction?

Temperature scaling makes confidence values more honest on average but promises nothing. Conformal prediction gives sets with a coverage guarantee. They solve different problems and are used together.

Q. What does Grad-CAM compute?

The gradient of the class score with respect to the last conv feature maps, averaged per map as importance weights, a weighted sum of the maps, ReLU, then upsampled to image size.

Q. Is your model trustworthy?

Partly. Bacterial decisions are anatomically grounded, but the audit shows viral decisions partly use image borders, so I report that as a limitation and propose lung segmentation.

Q. Tell me about a hard bug.

The app and the evaluation disagreed on the same image. I ruled out preprocessing (identical pixels), isolated the difference to the Apple GPU's graph mode, traced it to the tensorflow-metal graph optimizer, fixed it by disabling that optimizer, and retrained every model.

Q. What would you do next?

A separate calibration split, an external test set, lung segmentation, and chest-X-ray-pretrained backbones.

14. How to run the project

```
git clone https://github.com/sumankit/PneumoScope-XAI.git
cd PneumoScope-XAI
python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

python -m src.data.make_manifest --download     # dataset (~2.3 GB) + split
python -m src.data.eda
python -m src.models.baseline_ml
python -m src.models.train_dl --model cnn
python -m src.models.train_dl --model densenet
for r in 224 384 512; do python -m src.models.train_dl --model effnetv2 --img-size $r; done
python -m src.evaluation.compare                # picks the deployed model
python -m src.evaluation.resolution_study
python -m src.evaluation.conformal
python -m src.evaluation.gradcam_report

streamlit run app/streamlit_app.py              # the app
pytest -q                                       # 13 unit tests
```

Or `./run_all.sh`. On the Apple M5 the full pipeline takes about 3.5–4 hours. The trained deployed model is already in the repository, so the app runs without training.

15. Glossary

Term	Meaning
Augmentation	Random realistic changes to training images (rotate, zoom...) to reduce overfitting
Backbone	The pretrained convolutional part of a network, without its classification head
Batch / epoch	A batch is a group of images processed together; an epoch is one pass over all training images
Calibration	How well stated confidence matches actual accuracy
CLAHE	Contrast-Limited Adaptive Histogram Equalisation: local contrast enhancement
Conformal set	Set of labels guaranteed to contain the truth with probability ≥ 1 − α
Coverage	Fraction of cases whose true label is inside the prediction set
ECE	Expected Calibration Error
Fine-tuning	Continuing to train some pretrained layers with a small learning rate
FLOPs	Floating-point operations: a measure of compute per image
Grad-CAM	Heatmap of the image regions that drove a prediction
Logit	A raw score before softmax
Macro-F1	Average of per-class F1 scores
McNemar test	Paired significance test between two classifiers on the same data
Overfitting	Memorising training data instead of learning general patterns
q̂ (q-hat)	The conformal threshold computed from calibration scores
Shortcut learning	A model using irrelevant cues (markers, borders) that happen to correlate with the label
Softmax	Function that turns scores into probabilities summing to 1
Transfer learning	Starting from weights learned on another task
Validation set	Data used to tune decisions during development; never used to fit weights